UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

Dipartimento di Scienze Fisiche,
Informatiche e Matematiche

14. Compilatori e Parallel Programming Models

Linguaggi e Compilatori [I215-011]

Corso di Laurea in INFORMATICA
(D.M.270/04) [16-262]
Anno accademico 2022/2023

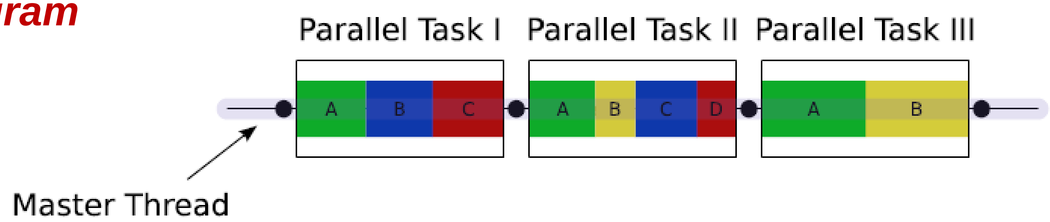
Prof. Andrea Marongiu
andrea.marongiu@unimore.it

Programming model: OpenMP

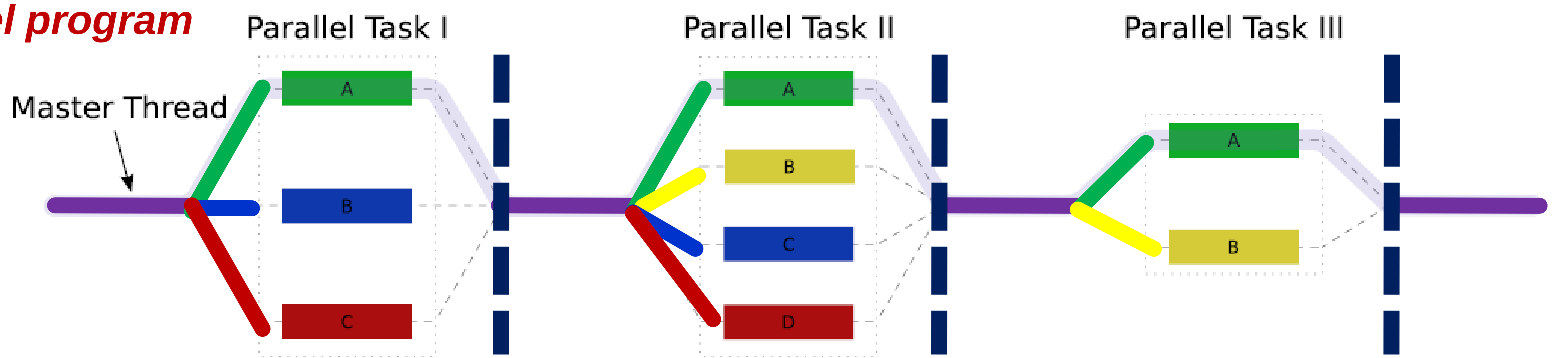
- De-facto standard for the shared memory programming model
- A collection of compiler directives, library routines and environment variables
- Easy to specify parallel execution within a serial code
- Targets several types of parallelism (data, task, heterogeneous)
- Popular in high-end embedded
- Requires special support in the compiler
- Generates calls to threading libraries (e.g. pthreads)

Fork/Join Parallelism

Sequential program



Parallel program



- Initially only master thread is active
- Master thread executes sequential code
- Fork: Master thread creates or awakens additional threads to execute parallel code
- Join: At the end of parallel code created threads are suspended upon **barrier** synchronization

Pragmas

- Pragma: a compiler directive in C or C++
- Stands for "pragmatic information"
- A way for the programmer to communicate with the compiler
- Compiler free to ignore pragmas: original sequential semantic is not altered
- Syntax:

  #**pragma omp** <*rest of pragma*>

Components of OpenMP
a subset of the directives

Directives

- Parallel regions
  - ***#pragma omp parallel***
- Work sharing
  - ***#pragma omp for***
  - ***#pragma omp sections***
- Synchronization
  - ***#pragma omp barrier***
  - ***#pragma omp critical***
  - ***#pragma omp atomic***

Clauses

- Data scope attributes
  - ***private***
  - ***shared***
  - ***reduction***
- Loop scheduling
  - ***static***
  - ***dynamic***

Runtime Library

- Thread Forking/Joining
  - ***omp_parallel_start()***
  - ***omp_parallel_end()***
- Loop scheduling
- Thread IDs
  - ***omp_get_thread_num()***
  - ***omp_get_num_threads()***

Outlining parallelism
The **parallel** directive

```
void ex1 ()
{
  <bb 2> [0.00%]:
  __builtin_GOMP_parallel (
       ex1._omp_fn.0, 0B, 0, 0);
}

ex1._omp_fn.0 (void * .omp_data_i)
{
  <bb 3> [0.00%]:
  printf ("Hello World!\n");
  return;
}
```

hello.c.012t.ompexp

Code originally contained within the scope of the pragma is outlined to a new function within the compiler

```
void ex1 ()
{
#pragma omp parallel
    {
      printf ("\nHello world!");
    }
}
```

A sequential program..
..is easily parallelized

- Fundamental construct to outline parallel computation within a sequential program
- Code within its scope is replicated among threads
- Defers implementation of parallel execution to the runtime (machine-specific, e.g. *pthread_create*)

Outlining parallelism
The **parallel** directive

```
void ex1 ()
{
  <bb 2> [0.00%]:
  __builtin_GOMP_parallel (
       ex1._omp_fn.0, 0B, 0, 0);
}


ex1._omp_fn.0 (void * .omp_data_i)
{
  <bb 3> [0.00%]:
  printf ("Hello World!\n");
  return;
}
```

hello.c.012t.ompexp

The #pragma construct in the **main** function is replaced with function calls to the runtime library

```
void ex1 ()
{
#pragma omp parallel
   {
     printf (“\nHello world!”);
   }
}
```

NOTE: The **master** thread is the only thread that executes this code

#pragma omp parallel

Within the runtime we first call a function to fork new threads,

```
void ex1 ()
{
  <bb 2> [0.00%]:
  __builtin_GOMP_parallel (
        ex1._omp_fn.0, 0B, 0, 0);
}


ex1._omp_fn.0 (void * .omp_data_i)
{
  <bb 3> [0.00%]:
  printf ("Hello World!\n");
  return;
}
```

hello.c.012t.ompexp

GOMP_parallel_start

```
void
GOMP_parallel (void (*fn) (void *),
               void *data,
               unsigned num_threads,
               unsigned int flags)
{
  num_threads = ... num_threads ...;
  gomp_team_start (fn, data,
                   num_threads, ...);
  fn (data);
  ialias_call (GOMP_parallel_end) ();
}
```

```
void ex1 ()
{
#pragma omp parallel
  {
    printf ("\nHello world!");
  }
}
```

#pragma omp parallel

```
void ex1 ()
{
  <bb 2> [0.00%]:
  __builtin_GOMP_parallel (
        ex1._omp_fn.0, 0B, 0, 0);
}


ex1._omp_fn.0 (void * .omp_data_i)
{
  <bb 3> [0.00%]:
  printf ("Hello World!\n");
  return;
}
```

hello.c.012t.ompexp

```
void ex1 ()
{
#pragma omp parallel
   {
     printf ("\nHello world!");
   }
}
```

Within the runtime we first call a function to fork new threads, **passing a pointer to the outlined function**

GOMP_parallel_start

```
void
GOMP_parallel (void (*fn) (void *),
               void *data,
               unsigned num_threads,
               unsigned int flags)
{
  num_threads = ... num_threads ...;
  gomp_team_start (fn, data,
                   num_threads, ...);
  fn (data);
  ialias_call (GOMP_parallel_end) ();
}
```

…and **data**, which we will be looking at in a moment…

#pragma omp parallel

```
void ex1 ()
{
  <bb 2> [0.00%]:
  __builtin_GOMP_parallel (
       ex1._omp_fn.0, 0B, 0, 0);
}


ex1._omp_fn.0 (void * .omp_data_i)
{
  <bb 3> [0.00%]:
  printf ("Hello World!\n");
  return;
}
```

hello.c.012t.ompexp

```
void ex1 ()
{
#pragma omp parallel
    {
      printf ("\nHello world!");
    }
}
```

Then the master itself calls the parallel function

```
void
GOMP_parallel (void (*fn) (void *),
               void *data,
               unsigned num_threads,
               unsigned int flags)
{
  num_threads = ... num_threads ...;
  gomp_team_start (fn, data,
                   num_threads, ...);
  fn (data);
  ialias_call (GOMP_parallel_end) ();
}
```

REMEMBER: The **master** thread is the only thread that executes this library code

#pragma omp parallel

```
void ex1 ()
{
  <bb 2> [0.00%]:
  __builtin_GOMP_parallel (
       ex1._omp_fn.0, 0B, 0, 0);
}


ex1._omp_fn.0 (void * .omp_data_i)
{
  <bb 3> [0.00%]:
  printf ("Hello World!\n");
  return;
}
```

hello.c.012t.ompexp

```
void ex1 ()
{
#pragma omp parallel
   {
     printf ("\nHello world!");
   }
}
```

Finally we call the runtime to synchronize threads with a barrier and suspend them

```
void
GOMP_parallel (void (*fn) (void *),
               void *data,
               unsigned num_threads,
               unsigned int flags)
{
  num_threads = ... num_threads ...;
  gomp_team_start (fn, data,
                   num_threads, ...);
  fn (data);
  ialias_call (GOMP_parallel_end) ();
}
```

REMEMBER: The **master** thread is the only thread that executes this library code

GOMP: The OpenMP runtime library

- Where does this library code live?
- GCC implementation → **libgomp** https://github.com/gcc-mirror/gcc/tree/master/libgomp

```
hero-vm@ubuntu:~/hero-sdk/hero-gcc-toolchain/src/riscv-gcc/libgomp$ ls
acinclude.m4        hashtab.h          Makefile.in        parallel.c
aclocal.m4          icv.c              oacc-async.c       plugin
affinity.c          icv-device.c       oacc-cuda.c        priority_queue.c
alloc.c             iter.c             oacc-host.c        priority_queue.h
atomic.c            iter_ull.c         oacc-init.c        sections.c
barrier.c           libgomp_f.h.in     oacc-int.h         single.c
ChangeLog           libgomp_g.h        oacc-mem.c         splay-tree.c
ChangeLog.graphite  libgomp.h          oacc-parallel.c    splay-tree.h
config              libgomp.map        oacc-plugin.c      target.c
config.h.in         libgomp-plugin.c   oacc-plugin.h      task.c
configure           libgomp-plugin.h   omp.h.in           taskloop.c
configure.ac        libgomp.spec.in    omp_lib.f90.in     team.c
configure.tgt       libgomp.texi       omp_lib.h.in       testsuite
critical.c          lock.c             openacc.f90        work.c
env.c               loop.c             openacc.h
error.c             loop_ull.c         openacc_lib.h
fortran.c           Makefile.am        ordered.c
```

Easy to spot at least one file per #pragma construct

GOMP: The OpenMP runtime library

- Where does this library code live?
- GCC implementation → **libgomp**
  https://github.com/gcc-mirror/gcc/tree/master/libgomp

```
hero-vm@ubuntu:~/hero-sdk/hero-gcc-toolchain/src/riscv-gcc/libgomp$ ls
acinclude.m4        hashtab.h          Makefile.in         parallel.c
aclocal.m4          icv.c              oacc-async.c        plugin
affinity.c          icv-device.c       oacc-cuda.c         priority_queue.c
alloc.c             iter.c             oacc-host.c         priority_queue.h
atomic.c            iter_ull.c         oacc-init.c         sections.c
barrier.c           libgomp_f.h.in     oacc-int.h          single.c
ChangeLog           libgomp_g.h        oacc-mem.c          splay-tree.c
ChangeLog.graphite  libgomp.h          oacc-parallel.c     splay-tree.h
config              libgomp.map        oacc-plugin.c       target.c
config.h.in         libgomp-plugin.c   oacc-plugin.h       task.c
configure           libgomp-plugin.h   omp.h.in            taskloop.c
configure.ac        libgomp.spec.in    omp_lib.f90.in      team.c
configure.tgt       libgomp.texi       omp_lib.h.in        testsuite
critical.c          lock.c             openacc.f90         work.c
env.c               loop.c             openacc.h
error.c             loop_ull.c         openacc_lib.h
fortran.c           Makefile.am        ordered.c
```

Easy to spot at least one file per #pragma construct

#pragma omp parallel

Data scope attributes

```
void ex2 ()
{
  int id;
  int base = 5;
  int slaves = 0;

  #pragma omp parallel
  {
    id = omp_get_thread_num();
    slaves = omp_get_num_threads() - 1;

    if (id == 0)
      printf ("Master: base = %d.", base*slaves);
    else
      printf ("Slave: base = %d.", base*id);
  }
}
```

Call runtime to get thread ID:
Every thread sees a different value

Master and slave threads access the same variable **a**

A slightly more complex example

#pragma omp parallel

Data scope attributes

```
void ex2 ()
{
  int id;
  int base = 5;
  int slaves = 0;

  #pragma omp parallel
  {
    id = omp_get_thread_num();
    slaves = omp_get_num_threads() - 1;

    if (id == 0)
      printf ("Master: base = %d.", base*slaves);
    else
      printf ("Slave: base = %d.", base*id);
  }
}
```

Call runtime to get thread ID:
Every thread sees a different value

How to inform the compiler about these different behaviors?

Master and slave threads access the same variable **a**

A slightly more complex example

#pragma omp parallel

Data scope attributes

```
void ex2 ()
{
  int id;
  int base = 5;
  int slaves = 0;

  #pragma omp parallel shared (base, slaves) private (id)
  {
    id = omp_get_thread_num();
    slaves = omp_get_num_threads() - 1;

    if (id == 0)
      printf ("Master: base = %d.", base*slaves);
    else
      printf ("Slave: base = %d.", base*id);
  }
}
```

Insert code to retrieve the address of the shared object from within each parallel thread

Allow symbol privatization: Each thread contains a private copy of this variable

A slightly more complex example

More data sharing clauses

- **firstprivate**
  - copyin, private storage
- **lastprivate**
  - Copyout, private storage

- **Exercise** – See how the generated code differs in the uses of a **shared** and a **firstprivate** variable

OpenMP lowering

Stampa l'intermedio dopo il passo di LOWERING di OpenMP

```
gcc -fopenmp -fdump-tree-omplower main.c
```

```
Introduced new external node (ex2._omp_fn.1/6).
ex2 ()
{
  int id;
  int base;
  int slaves;

  base = 5;
  slaves = 0;
  {
    .omp_data_o.2.base = base;
    .omp_data_o.2.slaves = slaves;
    #pragma omp parallel private(id) shared(slaves) firstprivate(base) [child fn: ex2._omp_fn.1 (.omp_data_o.2)]
      {
        .omp_data_i = (struct .omp_data_s.1 & restrict) &.omp_data_o.2;
        base = .omp_data_i->base;
        id = omp_get_thread_num ();
        D.6171 = omp_get_num_threads ();
        D.6189 = D.6171 + -1;
        .omp_data_i->slaves = D.6189;
        if (id == 0) goto <D.6186>; else goto <D.6187>;
        <D.6186>:
        D.6190 = .omp_data_i->slaves;
        D.6174 = base * D.6190;
        printf ("Master: base = %d.", D.6174);
        goto <D.6188>;
        <D.6187>:
        D.6176 = base * id;
        printf ("Slave: base = %d.", D.6176);
        <D.6188>:
        #pragma omp return
      }
    slaves = .omp_data_o.2.slaves;
    .omp_data_o.2 = {CLOBBER};
  }
}
```

hello.c.006t.omplower

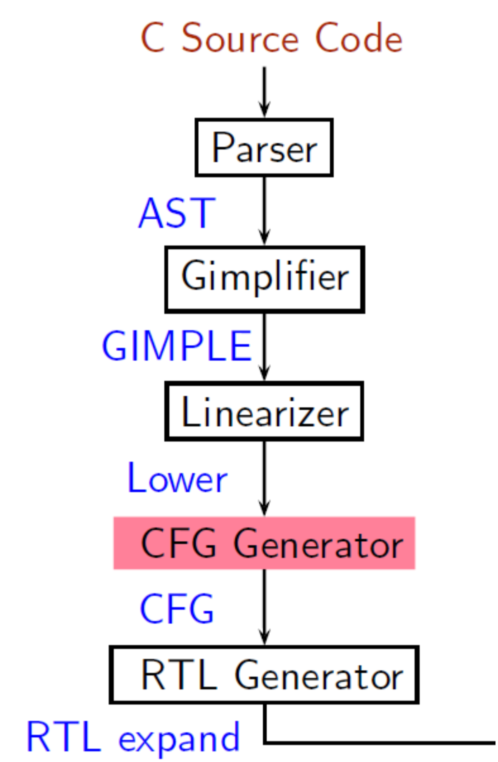
First of two stages of OpenMP processing:

1. **Lowering**
2. **Expansion**

Operates in high GIMPLE form

```
hello.c
hello.c.001t.tu
hello.c.002t.class
hello.c.003t.original
hello.c.004t.gimple
hello.c.006t.omplower
hello.c.007t.lower
hello.c.010t.eh
hello.c.011t.cfg
hello.c.012t.ompexp
```

ls -la



OpenMP lowering

```
Introduced new external node (ex2._omp_fn.1/6).
ex2 ()
{
  int id;
  int base;
  int slaves;

  base = 5;
  slaves = 0;
  {
    .omp_data_o.2.base = base;
    .omp_data_o.2.slaves = slaves;
    #pragma omp parallel private(id) shared(slaves) firstprivate(base) [child fn: ex2._omp_fn.1 (.omp_data_o.2)]
      {
        .omp_data_i = (struct .omp_data_s.1 & restrict) &.omp_data_o.2;
        base = .omp_data_i->base;
        id = omp_get_thread_num ();
        D.6171 = omp_get_num_threads ();
        D.6189 = D.6171 + -1;
        .omp_data_i->slaves = D.6189;
        if (id == 0) goto <D.6186>; else goto <D.6187>;
        <D.6186>:
        D.6190 = .omp_data_i->slaves;
        D.6174 = base * D.6190;
        printf ("Master: base = %d.", D.6174);
        goto <D.6188>;
        <D.6187>:
        D.6176 = base * id;
        printf ("Slave: base = %d.", D.6176);
        <D.6188>:
        #pragma omp return
      }
    slaves = .omp_data_o.2.slaves;
    .omp_data_o.2 = {CLOBBER};
  }
}
```

In this stage we create a new function node…

…and implement **data marshalling** for sharing data among parallel threads

(but outlining doesn't happen just yet)

hello.c.006t.omplower

OpenMP lowering

```
typedef struct
{
  int base;
  int slaves;
} omp_data_t
```

```
Introduced new external node (ex2._omp_fn.1/6).
ex2 ()
{
  int id;
  int base;
  int slaves;

  base = 5;
  slaves = 0;
  {
    .omp_data_o.2.base = base;
    .omp_data_o.2.slaves = slaves;
    #pragma omp parallel private(id) shared(slaves) firstprivate(base) [child fn: ex2._omp_fn.1 (.omp_data_o.2)]
      {
        .omp_data_i = (struct .omp_data_s.1 & restrict) &.omp_data_o.2;
        base = .omp_data_i->base;
        id = omp_get_thread_num ();
        D.6171 = omp_get_num_threads ();
        D.6189 = D.6171 + -1;
        .omp_data_i->slaves = D.6189;
        if (id == 0) goto <D.6186>; else goto <D.6187>;
        <D.6186>:
        D.6190 = .omp_data_i->slaves;
        D.6174 = base * D.6190;
        printf ("Master: base = %d.", D.6174);
        goto <D.6188>;
        <D.6187>:
        D.6176 = base * id;
        printf ("Slave: base = %d.", D.6176);
        <D.6188>:
        #pragma omp return
      }
    slaves = .omp_data_o.2.slaves;
    .omp_data_o.2 = {CLOBBER};
  }
}
```

marshalling IN/OUT (**omp_data_t omp_data_o;**)

…and implement **data marshalling** for sharing data among parallel threads

Replace uses (**omp_data_t omp_data_i;**)

hello.c.006t.omplower

OpenMP expansion

Stampa l'intermedio dopo il passo di EXPANSION di OpenMP

```
gcc -fopenmp -fdump-tree-ompexp main.c
```

```
ex2._omp_fn.1 (struct .omp_data_s.1 & restrict .omp_data_i)
{
  int slaves [value-expr: .omp_data_i->slaves]
  int D.6198;
  int D.6197;
  int D.6196;
  int D.6195;
  int D.6194;
  int id;
  int base;

  <bb 3> [0.00%]:
  base = .omp_data_i->base;
  id = omp_get_thread_num ();
  D.6194 = omp_get_num_threads ();
  D.6195 = D.6194 + -1;
  .omp_data_i->slaves = D.6195;
  if (id == 0)
    goto <bb 4>; [0.00%]
  else
    goto <bb 5>; [0.00%]

  <bb 6> [0.00%]:
  return;

  <bb 5> [0.00%]:
  D.6196 = base * id;
  printf ("Slave: base = %d.", D.6196);
  goto <bb 6>; [0.00%]

  <bb 4> [0.00%]:
  D.6197 = .omp_data_i->slaves;
  D.6198 = base * D.6197;
  printf ("Master: base = %d.", D.6198);
  goto <bb 6>; [0.00%]

}
```

hello.c.012t.ompexp

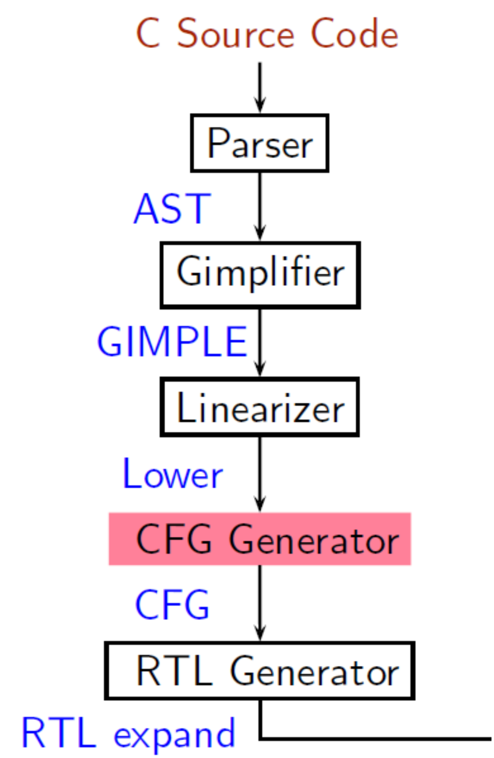
Second of two stages of OpenMP processing:

1. **Lowering**
2. **Expansion**

Operates in low GIMPLE form

```
hello.c
hello.c.001t.tu
hello.c.002t.class
hello.c.003t.original
hello.c.004t.gimple
hello.c.006t.omplower
hello.c.007t.lower
hello.c.010t.eh
hello.c.011t.cfg
hello.c.012t.ompexp
```

ls -la



OpenMP expansion

Stampa l'intermedio dopo il passo di EXPANSION di OpenMP

```
ex2._omp_fn.1 (struct .omp_data_s.1 & restrict .omp_data_i)
{
  int slaves [value-expr: .omp_data_i->slaves]
  int D.6198;
  int D.6197;
  int D.6196;
  int D.6195;
  int D.6194;
  int id;
  int base;

  <bb 3> [0.00%]:
  base = .omp_data_i->base;
  id = omp_get_thread_num ();
  D.6194 = omp_get_num_threads ();
  D.6195 = D.6194 + -1;
  .omp_data_i->slaves = D.6195;
  if (id == 0)
    goto <bb 4>; [0.00%]
  else
    goto <bb 5>; [0.00%]

  <bb 6> [0.00%]:
  return;

  <bb 5> [0.00%]:
  D.6196 = base * id;
  printf ("Slave: base = %d.", D.6196);
  goto <bb 6>; [0.00%]

  <bb 4> [0.00%]:
  D.6197 = .omp_data_i->slaves;
  D.6198 = base * D.6197;
  printf ("Master: base = %d.", D.6198);
  goto <bb 6>; [0.00%]

}
```

hello.c.012t.ompexp

```
gcc -fopenmp -fdump-tree-ompexp main.c
```

-fdump-tree-all per stampare la IR dopo ogni passo di otimizzazione del middle-end

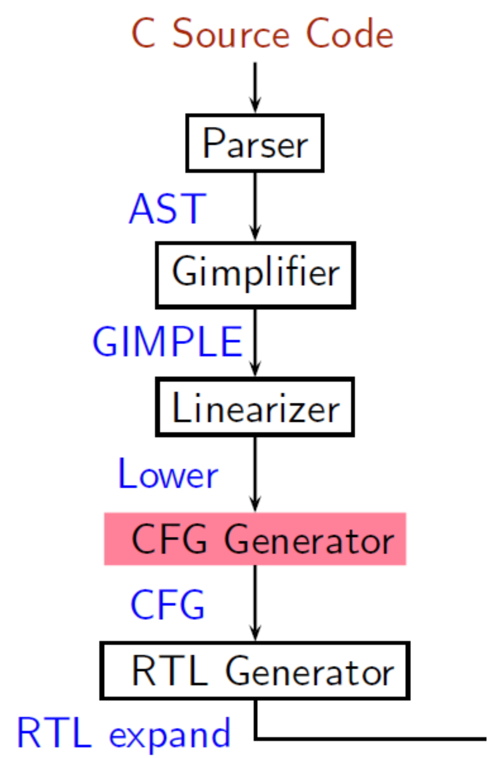
Second of two stages of OpenMP processing:

1. **Lowering**
2. **Expansion**

Operates in low GIMPLE form

```
hello.c
hello.c.001t.tu
hello.c.002t.class
hello.c.003t.original
hello.c.004t.gimple
hello.c.006t.omplower
hello.c.007t.lower
hello.c.010t.eh
hello.c.011t.cfg
hello.c.012t.ompexp
```

ls -la



OpenMP expansion

```
ex2._omp_fn.1 (struct .omp_data_s.1 & restrict .omp_data_i)
{
  int slaves [value-expr: .omp_data_i->slaves];
  int D.6198;
  int D.6197;
  int D.6196;
  int D.6195;
  int D.6194;
  int id;
  int base;

  <bb 3> [0.00%]:
  base = .omp_data_i->base;
  id = omp_get_thread_num ();
  D.6194 = omp_get_num_threads ();
  D.6195 = D.6194 + -1;
  .omp_data_i->slaves = D.6195;
  if (id == 0)
    goto <bb 4>; [0.00%]
  else
    goto <bb 5>; [0.00%]

  <bb 6> [0.00%]:
  return;

  <bb 5> [0.00%]:
  D.6196 = base * id;
  printf ("Slave: base = %d.", D.6196);
  goto <bb 6>; [0.00%]

  <bb 4> [0.00%]:
  D.6197 = .omp_data_i->slaves;
  D.6198 = base * D.6197;
  printf ("Master: base = %d.", D.6198);
  goto <bb 6>; [0.00%]

}
```

marshalling IN/OUT (**`omp_data_t omp_data_o;`**)

```
ex2 ()
{
  int slaves;
  int base;
  int id;
  struct .omp_data_s.1 .omp_data_o.2;
  int D.6190;
  int D.6189;

  <bb 2> [0.00%]:
  base = 5;
  slaves = 0;
  .omp_data_o.2.base = base;
  .omp_data_o.2.slaves = slaves;
  __builtin_GOMP_parallel (ex2._omp_fn.1, &.omp_data_o.2, 0, 0);
  slaves = .omp_data_o.2.slaves;
  .omp_data_o.2 = {CLOBBER};
  return;

}
```

Call runtime by passing a pointer to the outlined function and the marshalled data

Replace uses (**`omp_data_t omp_data_i;`**)

hello.c.012t.ompexp

Sharing work among threads
The **`for`** directive

- The **parallel** pragma instructs every thread to execute all of the code inside the block
- If we encounter a **for** loop that we want to divide among threads, we use the **for** pragma

```
#pragma omp for
```

#pragma omp for

#pragma omp for can be placed everywhere inside a **parallel** construct, or combined with it, as in the example

The code of the **for** loop is moved inside the outlined function.

```
void ex3 (int *a)
{
  int i;

#pragma omp parallel for
  for (i=0; i<10; i++)
    a[i] = i;
}
```

```
void ex3(...)
{
  omp_parallel_start(&parfun, ...);
  parfun();
  omp_parallel_end();
}

int parfun(...)
{
  int LB = ...;
  int UB = ...;

  for (i=LB; i<UB; i++)
      a[i] = i;
}
```

#pragma omp for

Every thread works on a different subset of the iteration space..

```
void ex3 (int *a)
{
  int i;

#pragma omp parallel for
  for (i=0; i<10; i++)
    a[i] = i;
}
```

```
void ex3(...)
{
  omp_parallel_start(&parfun, ...);
  parfun();
  omp_parallel_end();
}

int parfun(...)
{
  int LB = ...;
  int UB = ...;

  for (i=LB; i<UB; i++)
    a[i] = i;
}
```

#pragma omp for

..since lower and upper bounds (**LB**, **UB**) are computed locally as a function of the thread ID

```
void ex3(...)
{
  omp_parallel_start(&parfun, ...);
  parfun();
  omp_parallel_end();
}

int parfun(...)
{
  int LB = ...;
  int UB = ...;

  for (i=LB; i<UB; i++)
      a[i] = i;
}
```

```
void ex3 (int *a)
{
  int i;

#pragma omp parallel for
  for (i=0; i<10; i++)
    a[i] = i;
}
```

The **`schedule`** clause
Static Loop Partitioning

```
#pragma omp for schedule(static)
  {
    for (i=0; i<12; i++)
      a[i] = i;
  }
```

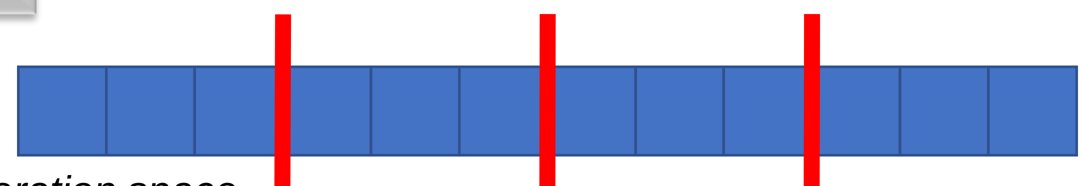
Useful for:
- *Simple, regular loops*
- *Iterations with equal duration*

Es. 12 iterations (N), 4 threads (Nthr)

DATA CHUNK

$$C = \mathit{ceil}\left(\frac{N}{Nthr}\right)$$

$$\frac{\textbf{3 iterations}}{\textbf{thread}}$$



		0	1	2	3
	Thread ID (TID)	**0**	**1**	**2**	**3**
LOWER BOUND	**LB** = C * TID	**0**	**3**	**6**	**9**
UPPER BOUND	**UB** = *min* { [C * (TID + 1)], N}	**3**	**6**	**9**	**12**

The `schedule` clause
Static Loop Partitioning

```
#pragma omp for schedule(static)
  {
    for (i=0; i<12; i++)
      a[i] = i;
  }
```

Es. 12 iterations (N), 4 threads (Nthr)

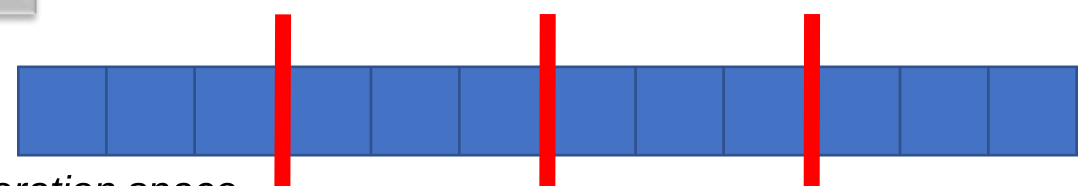
DATA CHUNK

$$C = \mathit{ceil}\left(\frac{N}{Nthr}\right)$$

$$\frac{\textbf{3 iterations}}{\textbf{thread}}$$

Useful for:

- *Simple, regular loops*
- *Iterations with equal duration*



Iteration space

		0	1	2	3
	Thread ID (TID)	**0**	**1**	**2**	**3**
LOWER BOUND	**LB** = C * TID	**0**	**3**	**6**	**9**
UPPER BOUND	**UB** = *min* { [C * (TID + 1)], N}	**3**	**6**	**9**	**12**

Static scheduling clauses

- **EXERCISE** – Generate and analyze the IR dumps for the OpenMP lowering and expansion passes for the **`#pragma omp for`** directive, playing with static chunking

- **`schedule (static, N)`**

The `schedule` clause
Static Loop Partitioning

Es. 12 iterations (N), 4 threads (Nthr)

```
#pragma omp for schedule(static)
  {
    for (i=0; i<12; i++)
      a[i] = i;
  }
```

DATA CHUNK

eil (N / Nthr)

3 iterations / thread

Useful for:
- *Simple, regular loo*
- *Iterations with equa*

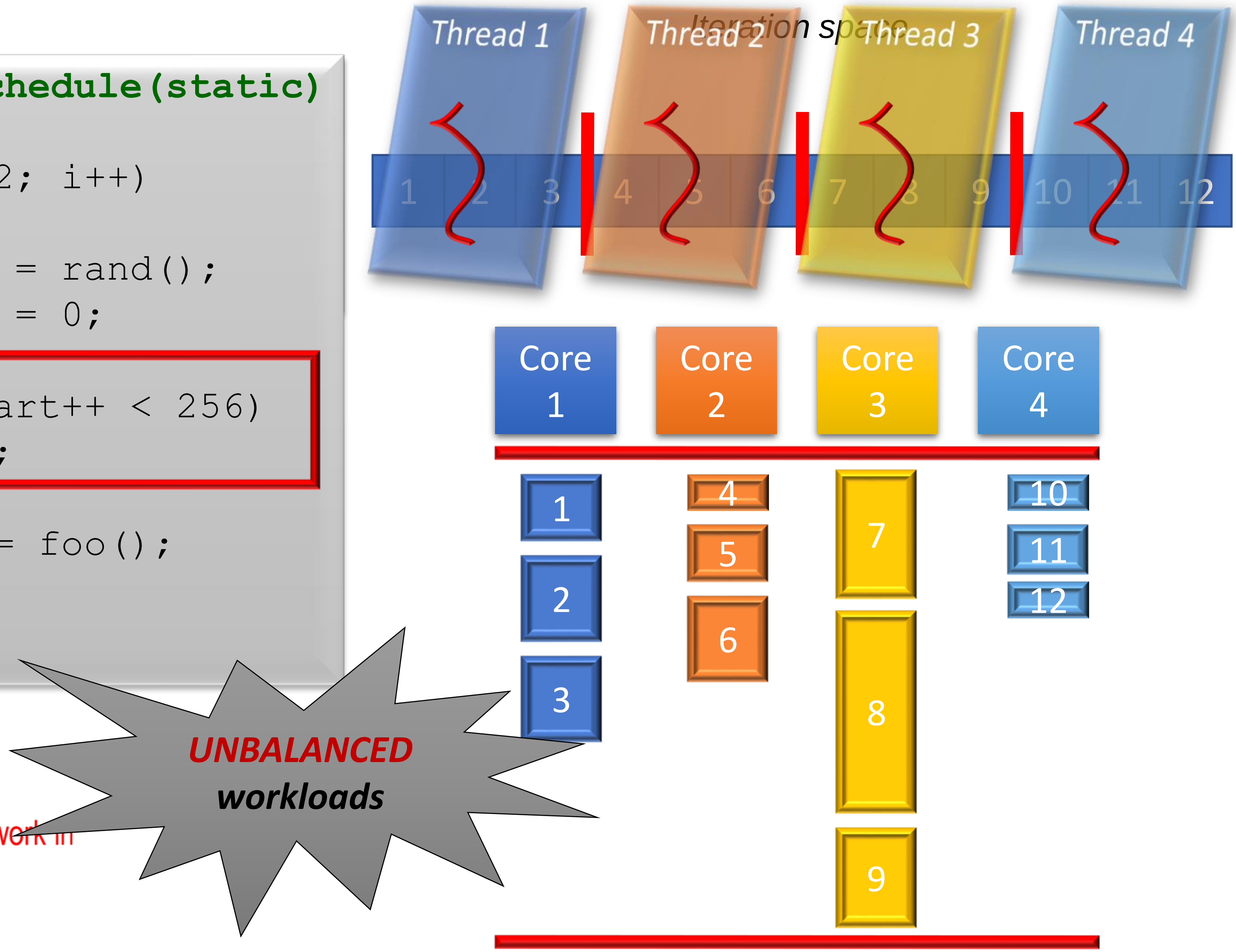
What happens with static scheduling when iterations have different duration?

	0	1	2	3
LOWER BO	0	3	6	9
UPPER BOUND (TID + 1)], N}	3	6	9	12

The **schedule** clause
Static Loop Partitioning

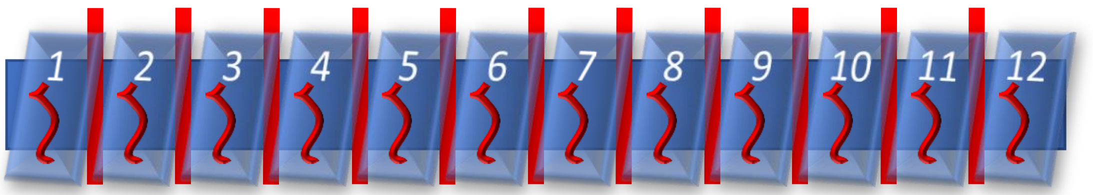
```
#pragma omp for schedule(static)
  {
    for (i=0; i<12; i++)
      {
        int start = rand();
        int count = 0;

        while (start++ < 256)
          count++;

        a[count] = foo();
      }
  }
```

A variable amount of work in each iteration



The **schedule** clause
Dynamic Loop Partitioning

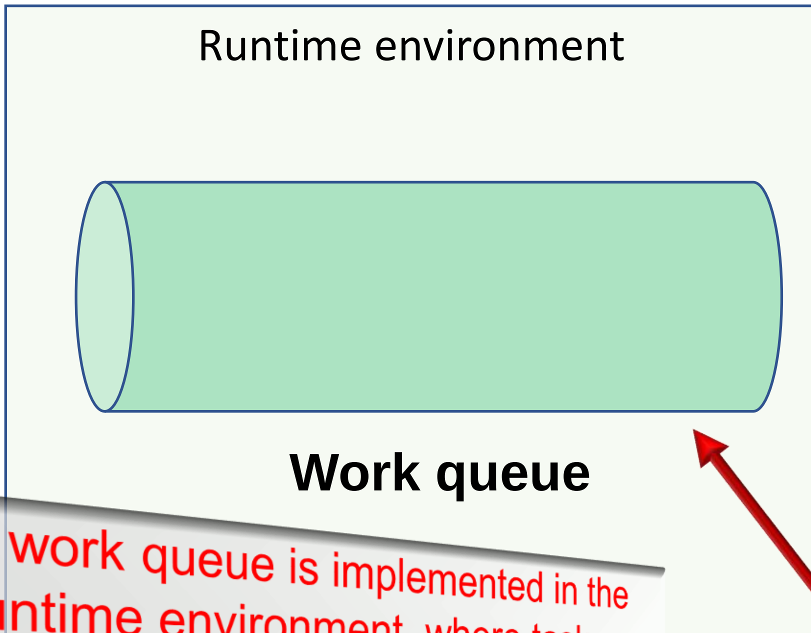
Iteration space



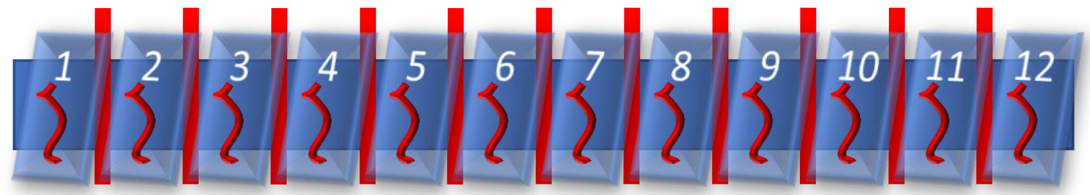

```
#pragma omp for schedule(dynamic)
  {
    for (i=0; i<12; i++)
      {
        int start = rand();
        int count = 0;

        while (start++ < 256)
          count++;

        a[count] = foo();
      }
  }
```

A thread is generated for every single iteration

The `schedule` clause
Dynamic Loop Partitioning



Iteration space

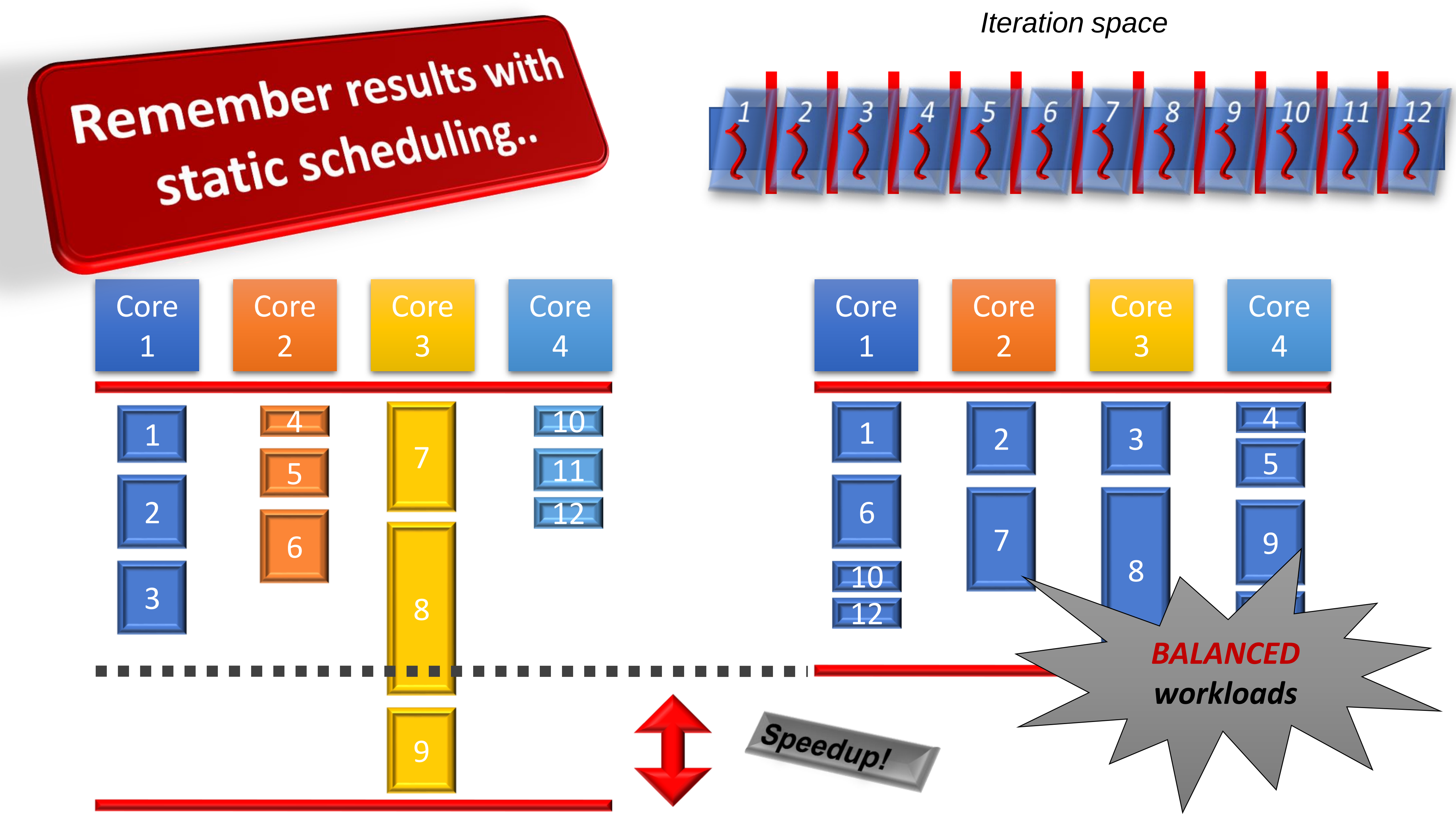


Work is fetched by threads from the runtime environment through synchronized accesses to the **work queue**

A work queue is implemented in the runtime environment, where tasks are stored

```
int parfun(…)
{
  int LB, UB;
  GOMP_loop_dynamic_next(&LB, &UB);

  for (i=LB; i<UB; i++) {…}
}
```

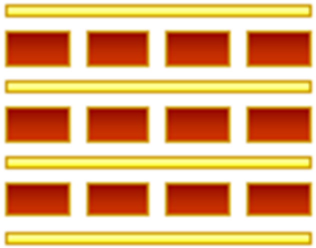
The **`schedule`** clause
Dynamic Loop Partitioning



Parallelization granularity
Iteration chunking

- Fine-grain Parallelism
  - Best opportunities for load balancing, but..
  - Small amounts of computational work between parallelism computation stages
  - Low computation to parallelization ratio ➔ High parallelization overhead

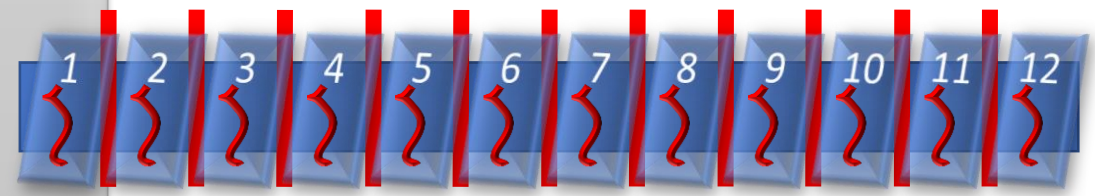
- Coarse-grain Parallelism
  - Harder to load balance efficiently, but..
  - Large amounts of computational work between parallelism computation stages
  - High computation to parallelization ratio ➔ Low parallelization overhead



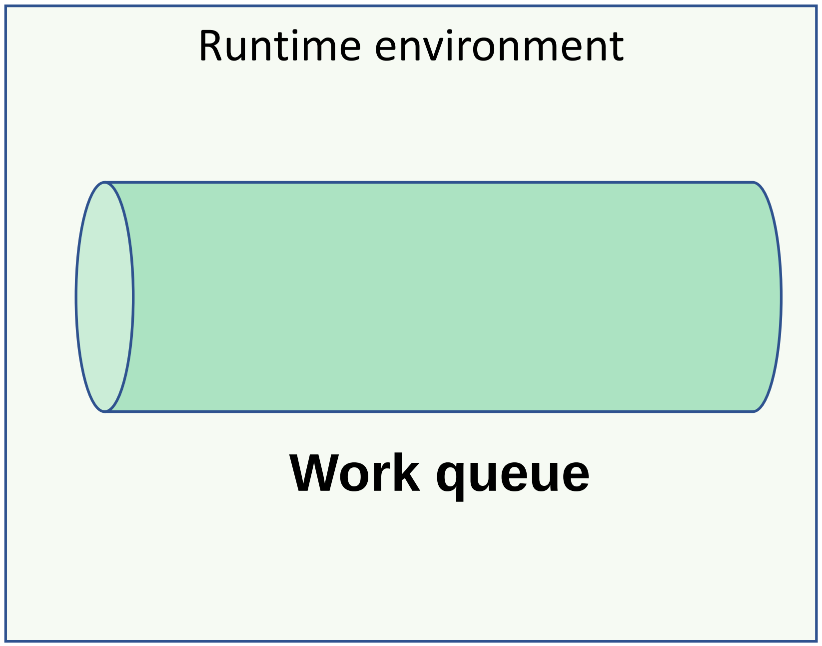
The **`schedule`** clause
Dynamic Loop Partitioning

```
#pragma omp for schedule(dynamic, 1)
  {
    for (i=0; i<12; i++)
      {
        int start = rand();
        int count = 0;

        while (start++ < 256)
          count++;

        a[count] = foo();
      }
  }
```

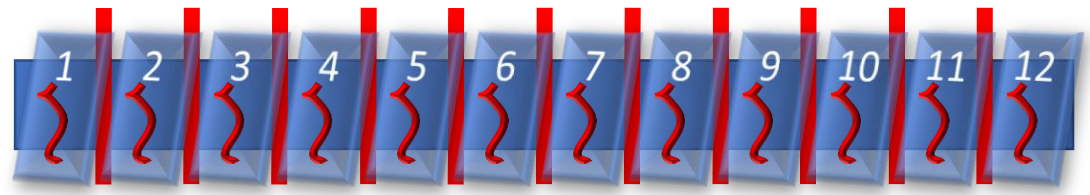
Iteration space



The **`schedule`** clause
Dynamic Loop Partitioning



Iteration space



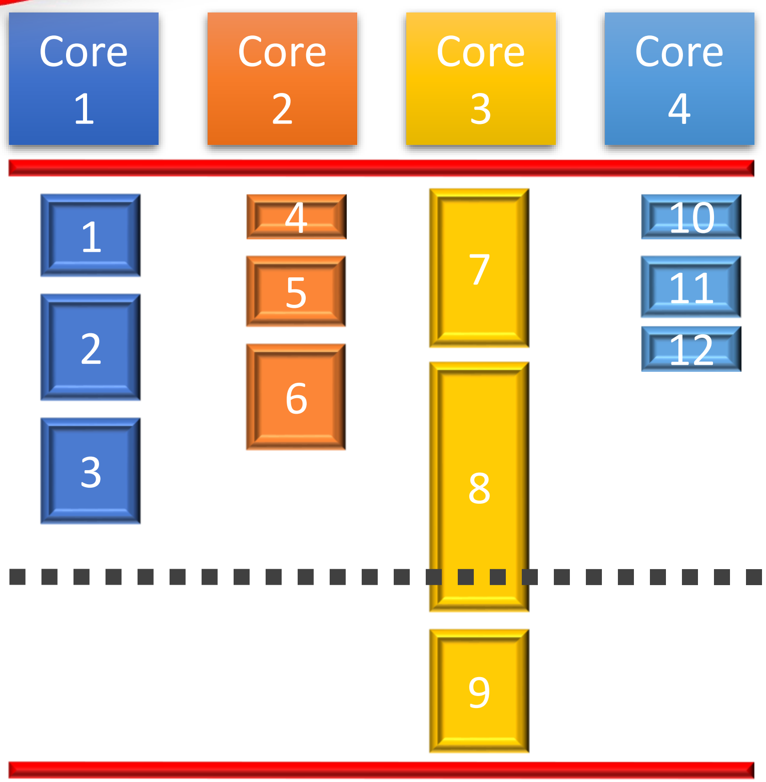
Runtime scheduling primitive is invoked at every iteration

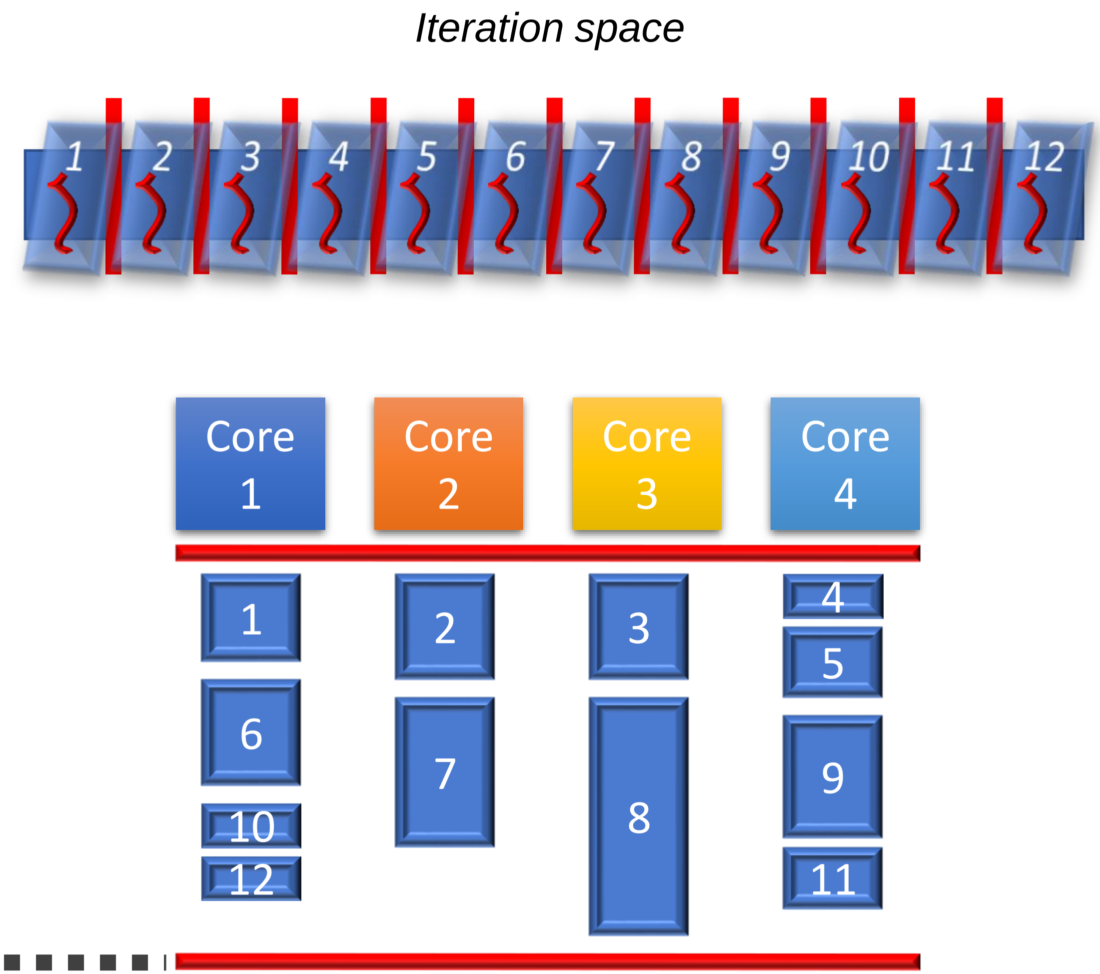
If granularity of WORK is very small the overhead for fine chunking is significant

```
int parfun(…)
{
  int LB, UB;
  GOMP_loop_dynamic_next(&LB, &UB);

  for (i=LB; i<UB; i++) {WORK}
}
```

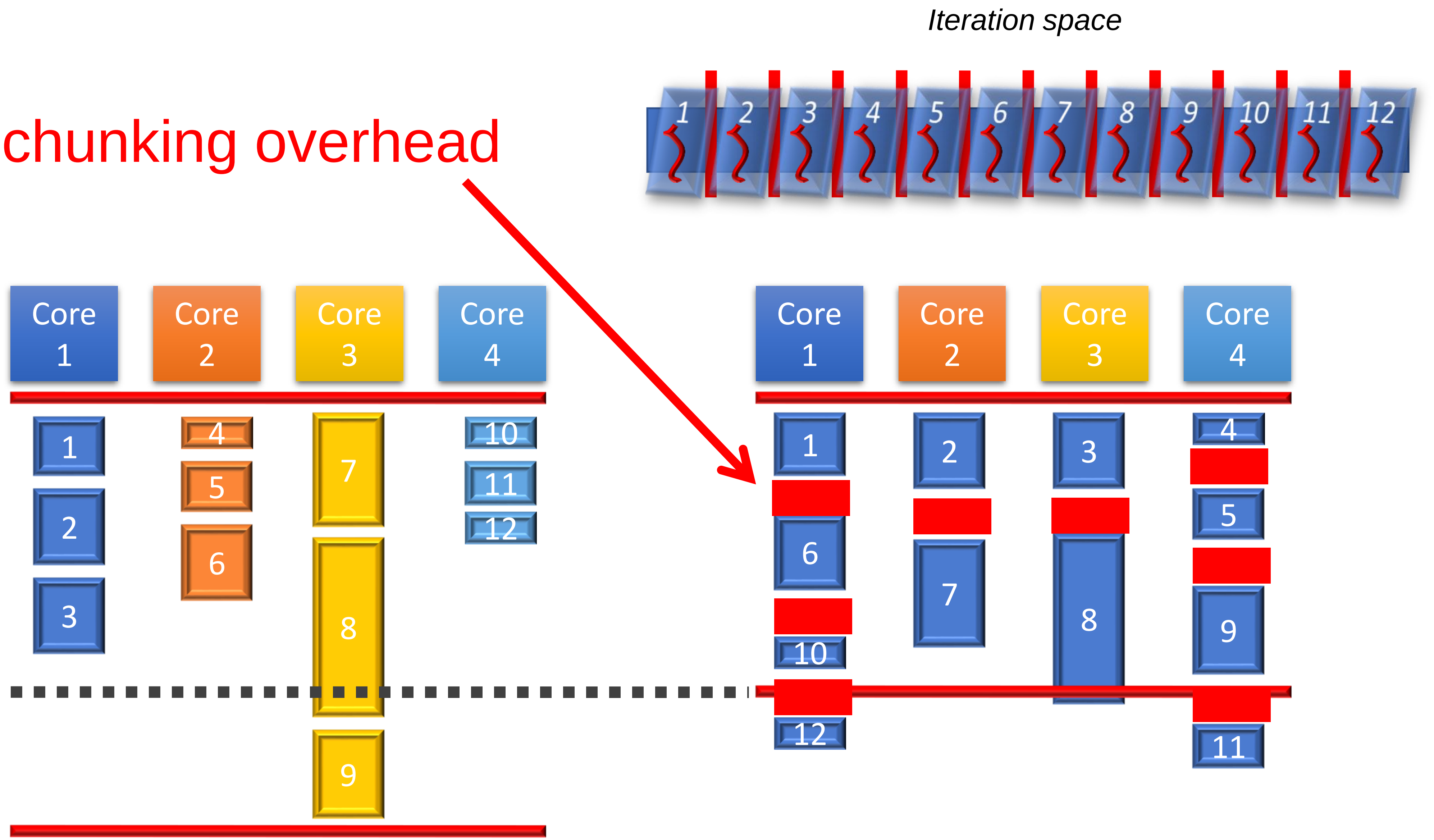
The **`schedule`** clause
Dynamic Loop Partitioning

Remember results with static scheduling..





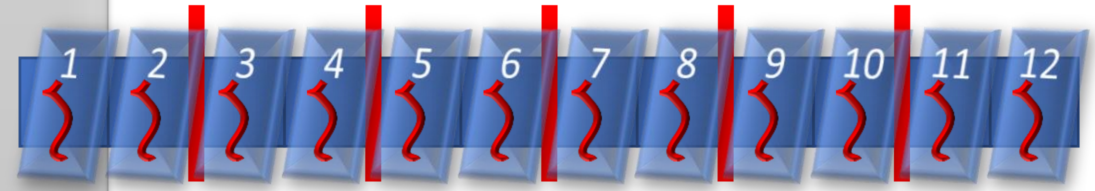
The `schedule` clause
Dynamic Loop Partitioning



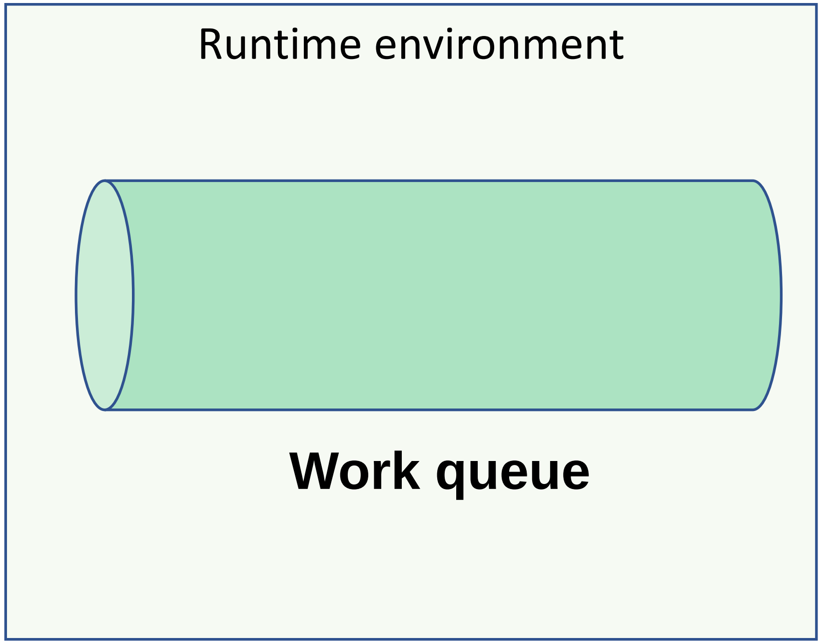
The **`schedule`** clause
Dynamic Loop Partitioning

Iteration space

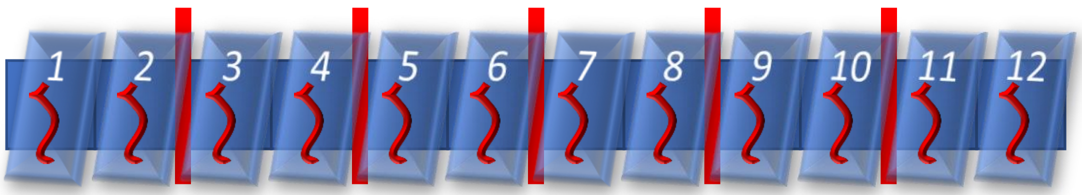
```
#pragma omp for schedule(dynamic, 2)
  {
    for (i=0; i<12; i++)
      {
        int start = rand();
        int count = 0;

        while (start++ < 256)
          count++;

        a[count] = foo();
      }
  }
```



The **schedule** clause
Dynamic Loop Partitioning



Iteration space

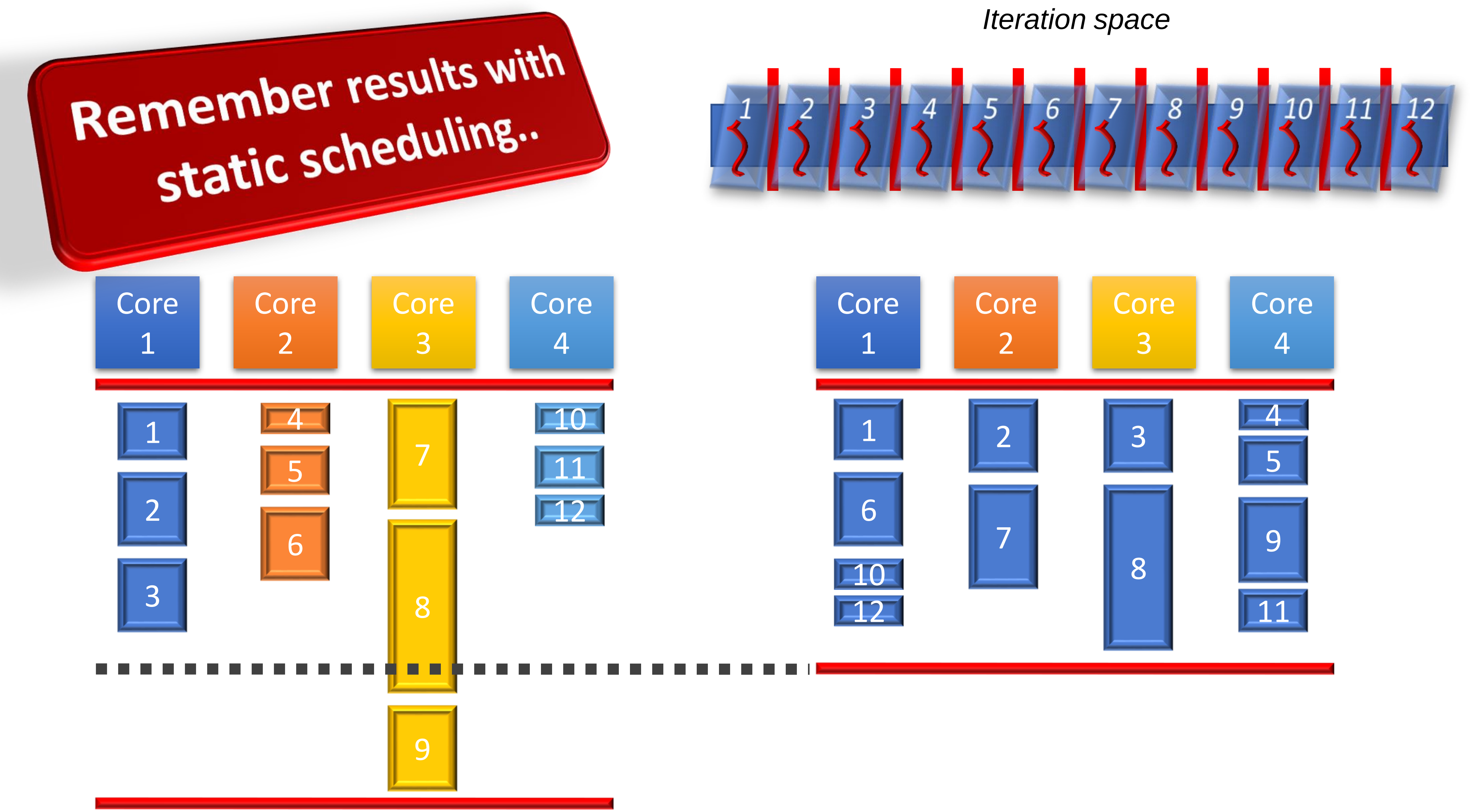


Runtime scheduling primitive is invoked every two iterations (half the times of previous case)

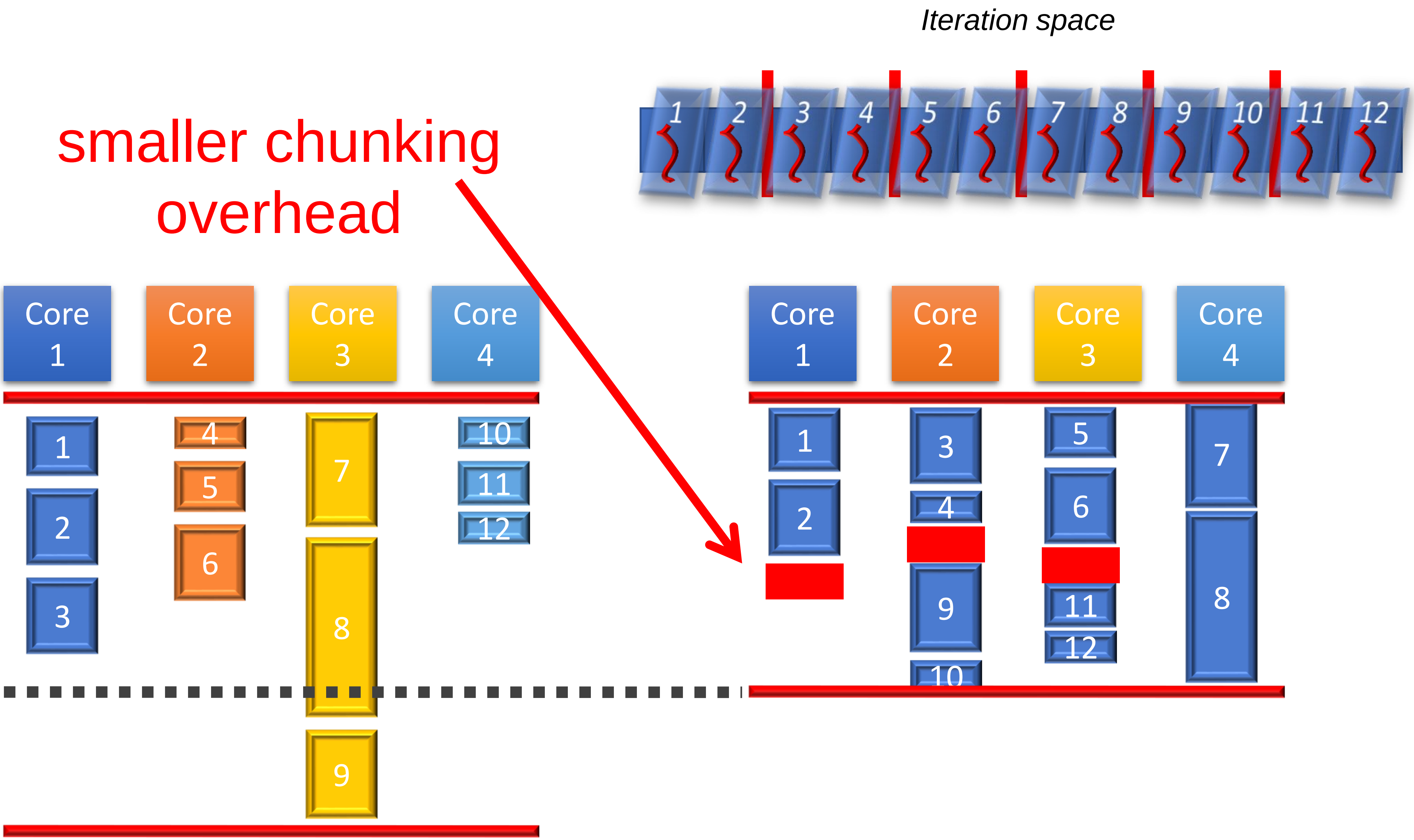
WORK is repeated twice among two scheduling events. Overhead gets amortized

```
int parfun(…)
{
  int LB, UB;
  GOMP_loop_dynamic_next(&LB, &UB);

  for (i=LB; i<UB; i++) {WORK}
}
```

The **`schedule`** clause
Dynamic Loop Partitioning



The **`schedule`** clause
Dynamic Loop Partitioning



More scheduling clauses

- **`schedule (guided`*`[, chunk]`*`)`**
  - Threads dynamically grab blocks of iterations. The size of the block starts large and shrinks down to size "chunk" as the calculation proceeds.
- **`schedule (runtime[, chunk])`**
  - Schedule and chunk size taken from the OMP_SCHEDULE environment variable (or the runtime library … for OpenMP 3.0)

- **EXERCISE** – Generate the IR dumps for the OpenMP lowering and expansion passes for different scheduling clauses

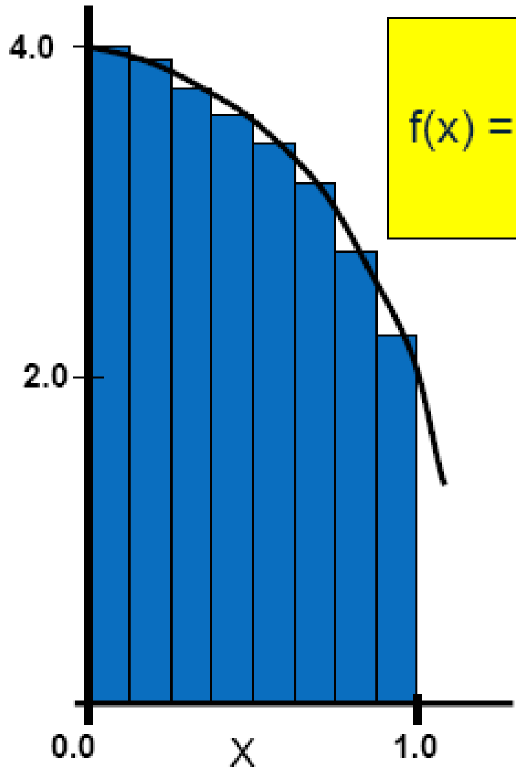
#pragma omp critical

- Critical Section: a portion of code that only one thread at a time may execute
- We denote a critical section by putting the pragma

```
#pragma omp critical
```

in front of a block of C code

π-finding code example



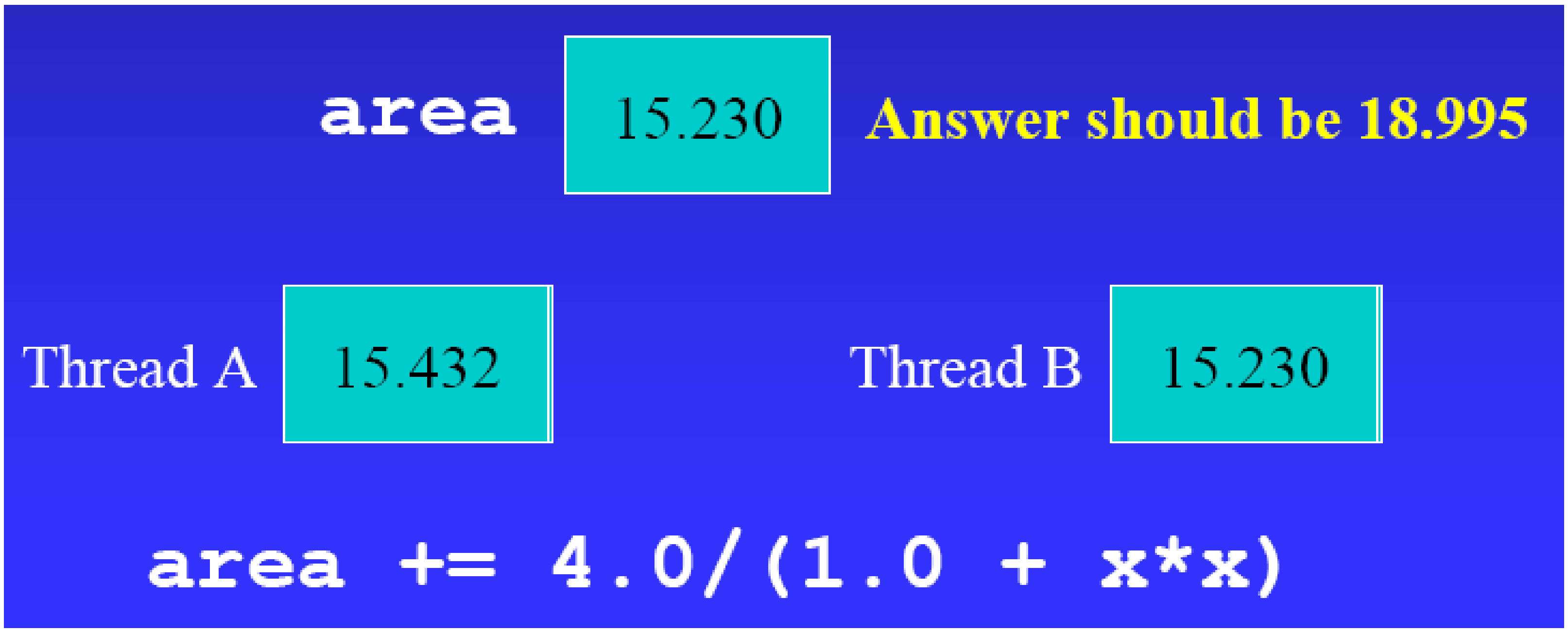
$$f(x) = \frac{4.0}{(1+x^2)}$$

```
double area, pi, x;
int i, n;
#pragma omp parallel for private(x) \
                         shared(area)
{
  for (i=0; i<n; i++) {
    x = (i + 0.5)/n;
    area += 4.0/(1.0 + x*x);
  }
}
  pi = area/n;
```

Synchronize accesses to shared variable **`area`** to avoid inconsistent results

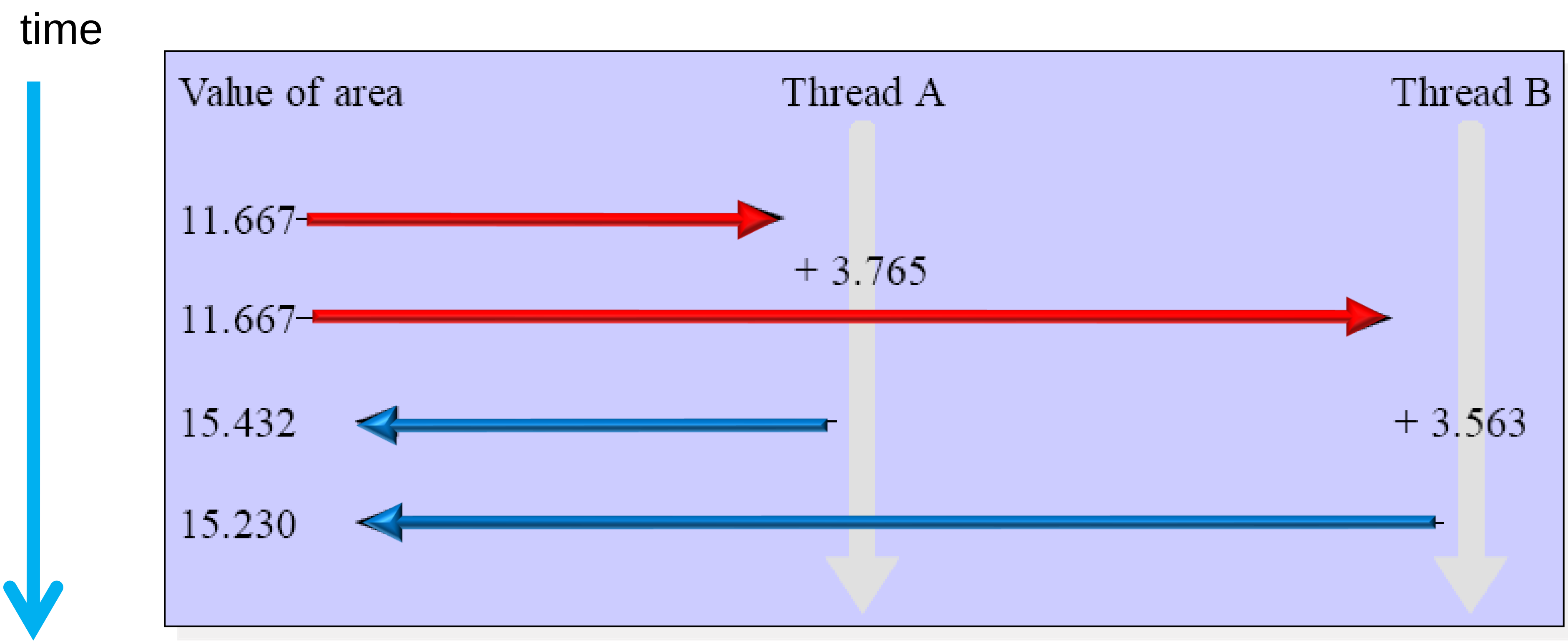
Race condition

- Ensure atomic updates of the shared variable **area** to avoid a race condition in which one process may "race ahead" of another and ignore changes



Race condition (Cont'd)

- *Thread A reads "11.667" into a local register*
- *Thread B reads "11.667" into a local register*
- *Thread A updates area with "11.667+3.765"*
- *Thread B ignores write from thread A and updates area with "11.667 + 3.563"*



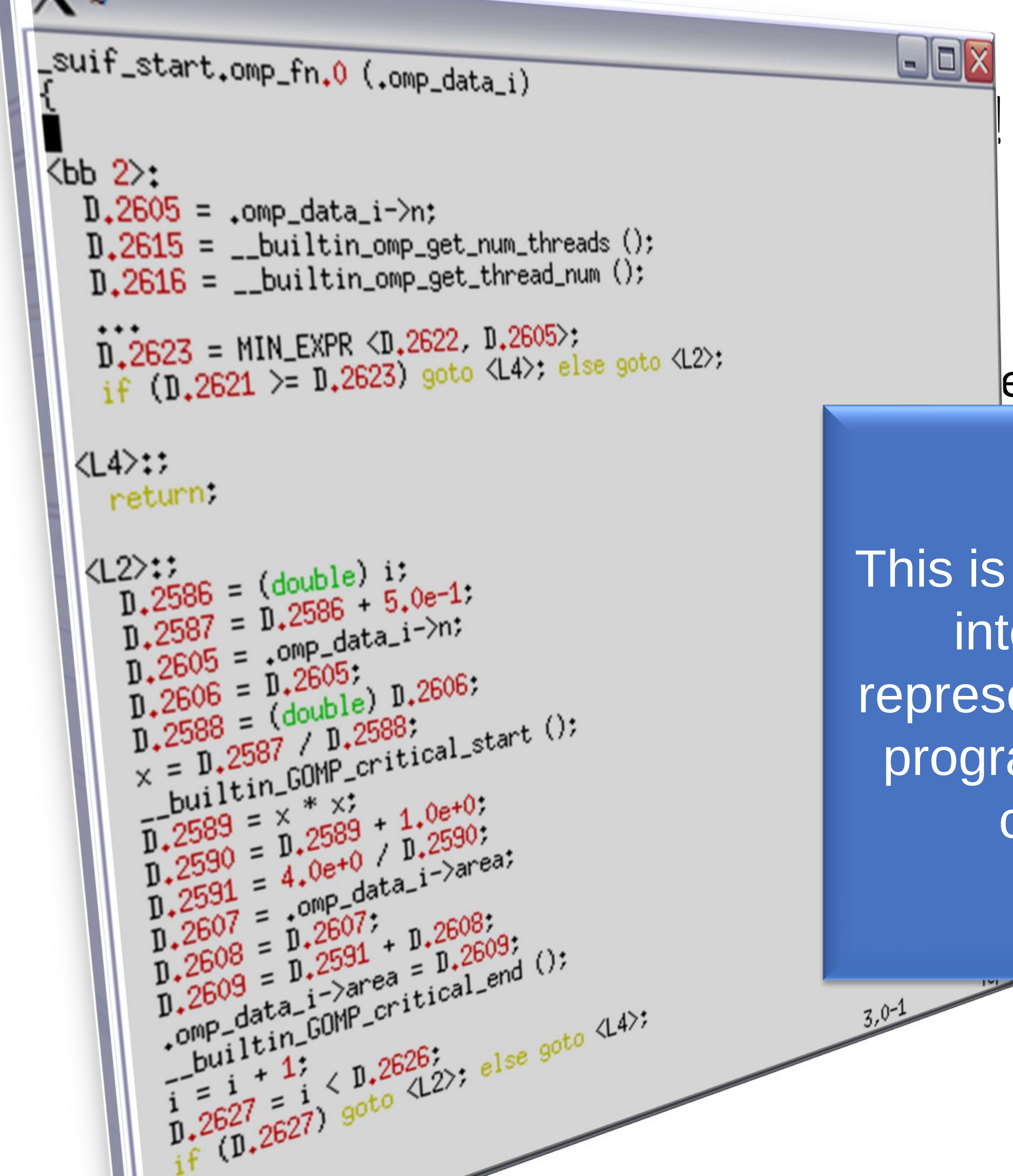
π-finding code example

```
double area, pi, x;
int i, n;
#pragma omp parallel for private(x) shared(area)
{
  for (i=0; i<n; i++) {
    x = (i +0.5)/n;
#pragma omp critical
    area += 4.0/(1.0 + x*x);
  }
}
  pi = area/n;
```

#pragma omp critical protects the code within its scope by acquiring a lock before entering the critical section and releasing it after execution

Correctness and performance!

- As a matter of fact, using locks makes execution sequential
- To dim this effect we should try use fine grained locking (i.e. make critical sections as small as possible)
- A simple instruction to compute the value of area in the previous example is translated into many more simpler instructions within the compiler!
- The programmer is not aware of the real granularity of the critical section



ecution sequential

.e.

e

This is a dump of the intermediate representation of the program within the compiler

```
_suif_start.omp_fn.0 (.omp_data_i)
{

<bb 2>:
  D.2605 = .omp_data_i->n;
  D.2615 = __builtin_omp_get_num_threads ();
  D.2616 = __builtin_omp_get_thread_num ();

  ...
  D.2623 = MIN_EXPR <D.2622, D.2605>;
  if (D.2621 >= D.2623) goto <L4>; else goto <L2>;

<L4>::
  return;

<L2>::
  D.2586 = (double) i;
  D.2587 = D.2586 + 5.0e-1;
  D.2605 = .omp_data_i->n;
  D.2605 = D.2605;
  D.2606 = (double) D.2606;
  D.2588 = (double) D.2606;
  x = D.2587 / D.2588;
  __builtin_GOMP_critical_start ();
  D.2589 = x * x;
  D.2590 = D.2589 + 1.0e+0;
  D.2591 = 4.0e+0 / D.2590;
  D.2607 = .omp_data_i->area;
  D.2607 = D.2607;
  D.2608 = D.2591 + D.2608;
  D.2609 = D.2608;
  .omp_data_i->area = D.2609;
  __builtin_GOMP_critical_end ();
  i = i + 1;
  D.2627 = i < D.2626;
  if (D.2627) goto <L2>; else goto <L4>;
```

ecution sequential

grained locking (i.e.

e)

e of area in the

more simpler

granularity of the

This is how the compiler represents the instruction

```
area += 4.0/(1.0 + x*x);
```

```
_suif_start.omp_fn.0 (.omp_data_i)
{

<bb 2>:
  D.2605 = .omp_data_i->n;
  D.2615 = __builtin_omp_get_num_threads ();
  D.2616 = __builtin_omp_get_thread_num ();
  ...
  D.2623 = MIN_EXPR <D.2622, D.2605>;
  if (D.2621 >= D.2623) goto <L4>; else goto <L2>;

<L4>:;
  return;

<L2>:;
  D.2586 = (double) i;
  D.2587 = D.2586 + 5.0e-1;
  D.2605 = .omp_data_i->n;
  D.2606 = D.2605;
  D.2588 = (double) D.2606;
  x = D.2587 / D.2588;
  __builtin_GOMP_critical_start ();
  D.2589 = x * x;
  D.2590 = D.2589 + 1.0e+0;
  D.2591 = 4.0e+0 / D.2590;
  D.2607 = .omp_data_i->area;
  D.2608 = D.2607;
  D.2609 = D.2591 + D.2608;
  .omp_data_i->area = D.2609;
  __builtin_GOMP_critical_end ();
  i = i + 1;
  D.2627 = i < D.2626;
  if (D.2627) goto <L2>; else goto <L4>;
                                                    3,0-1
```

call runtime to acquire lock

Lock-protected operations ***(critical section)***

call runtime to release lock

THIS IS DONE AT EVERY LOOP ITERATION!

ecution sequential

grained locking (i.e.

e)

e of area in the

more simpler

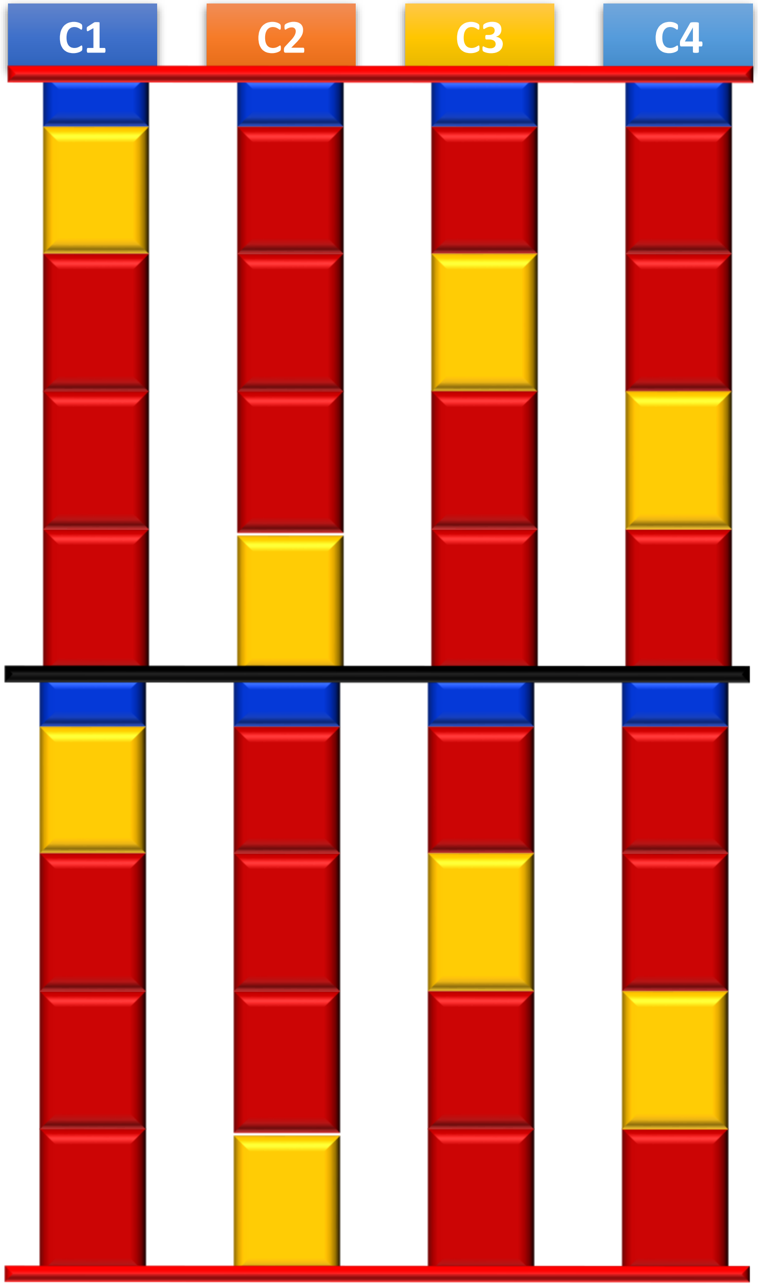
granularity of the

π-finding code example

```
double area, pi, x;
int i, n;
#pragma omp parallel for \
            private(x)    \
            shared(area)
{
  for (i=0; i<n; i++) {
    x = (i +0.5)/n;
  #pragma omp critical
    area += 4.0/(1.0 + x*x);
  }
}
  pi = area/n;
```

Parallel

Sequential



Waiting for lock

Correctness and performance!

- A programming pattern such as **`area += 4.0/(1.0 + x*x);`** in which we:
  - *Fetch the value of an operand*
  - *Add a value to it*
  - *Store the updated value*

  is called a reduction, and is commonly supported by parallel programming APIs

- OpenMP takes care of storing partial results in private variables and combining partial results after the loop

Correctness and performance!

```
double area, pi, x;
int i, n;
#pragma omp parallel for private(x) shared(area) reduction(+:area)
{
  for (i=0; i<n; i++) {
    x = (i +0.5)/n;

    area += 4.0/(1.0 + x*x);
  }
}
  pi = area/n;
```

The **reduction** clause instructs the compiler to create **private** copies of the **area** variable for every thread. At the end of the loop partial sums are combined on the shared **area** variable

C ance!

```
doub
int
#pra                                  shared(area) reduction(+:area)
{
   fo
}
}
   pi
```

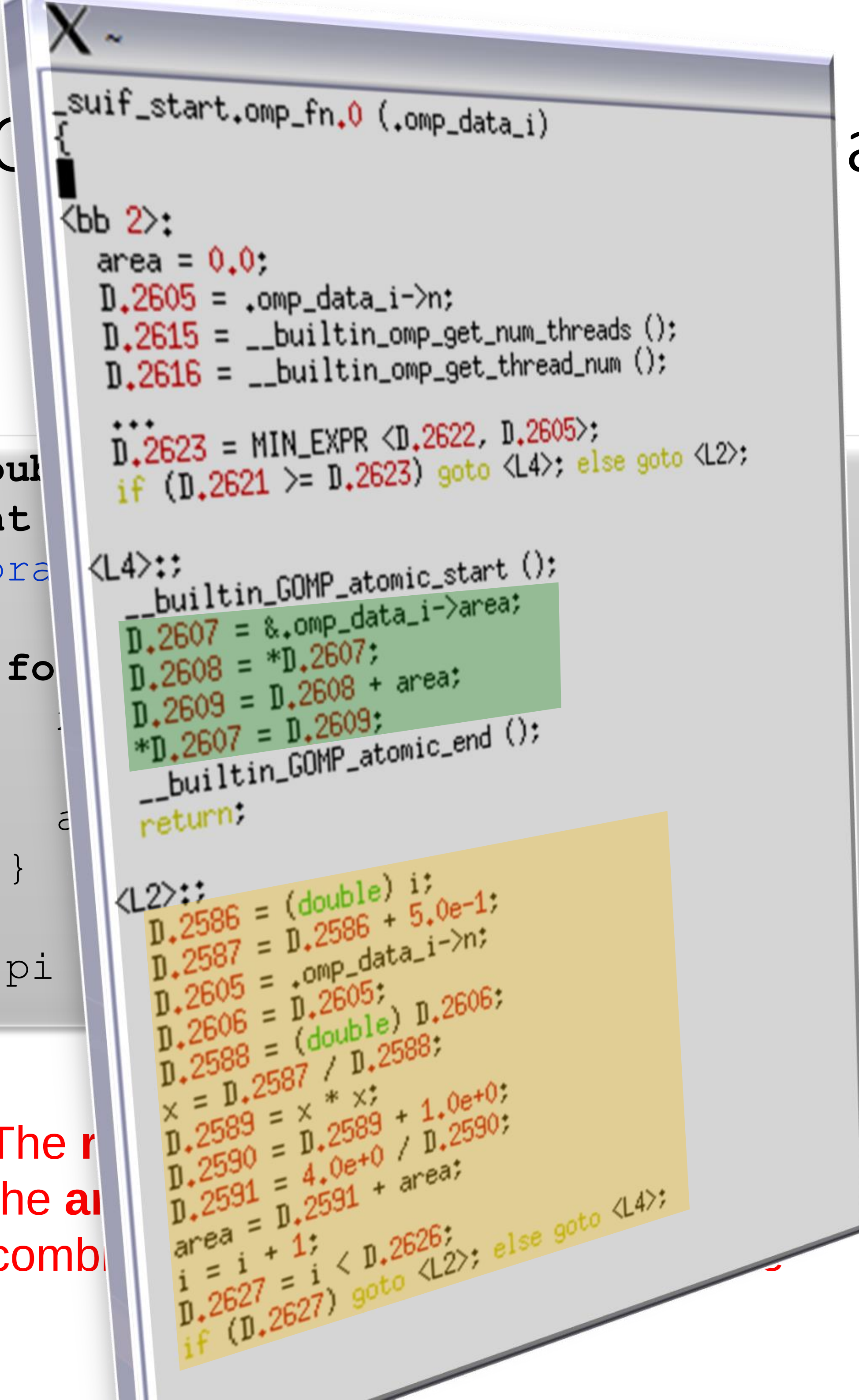
```
_suif_start.omp_fn.0 (.omp_data_i)
{

<bb 2>:
  area = 0.0;
  D.2605 = .omp_data_i->n;
  D.2615 = __builtin_omp_get_num_threads ();
  D.2616 = __builtin_omp_get_thread_num ();
  ...
  D.2623 = MIN_EXPR <D.2622, D.2605>;
  if (D.2621 >= D.2623) goto <L4>; else goto <L2>;

<L4>:;
  __builtin_GOMP_atomic_start ();
  D.2607 = &.omp_data_i->area;
  D.2608 = *D.2607;
  D.2609 = D.2608 + area;
  *D.2607 = D.2609;
  __builtin_GOMP_atomic_end ();
  return;

<L2>:;
  D.2586 = (double) i;
  D.2587 = D.2586 + 5.0e-1;
  D.2605 = .omp_data_i->n;
  D.2606 = D.2605;
  D.2588 = (double) D.2606;
  x = D.2587 / D.2588;
  D.2589 = x * x;
  D.2590 = D.2589 + 1.0e+0;
  D.2591 = 4.0e+0 / D.2590;
  area = D.2591 + area;
  i = i + 1;
  D.2627 = i < D.2626;
  if (D.2627) goto <L2>; else goto <L4>;
```

Shared variable is only updated at the end of the loop, when partial sums are computed

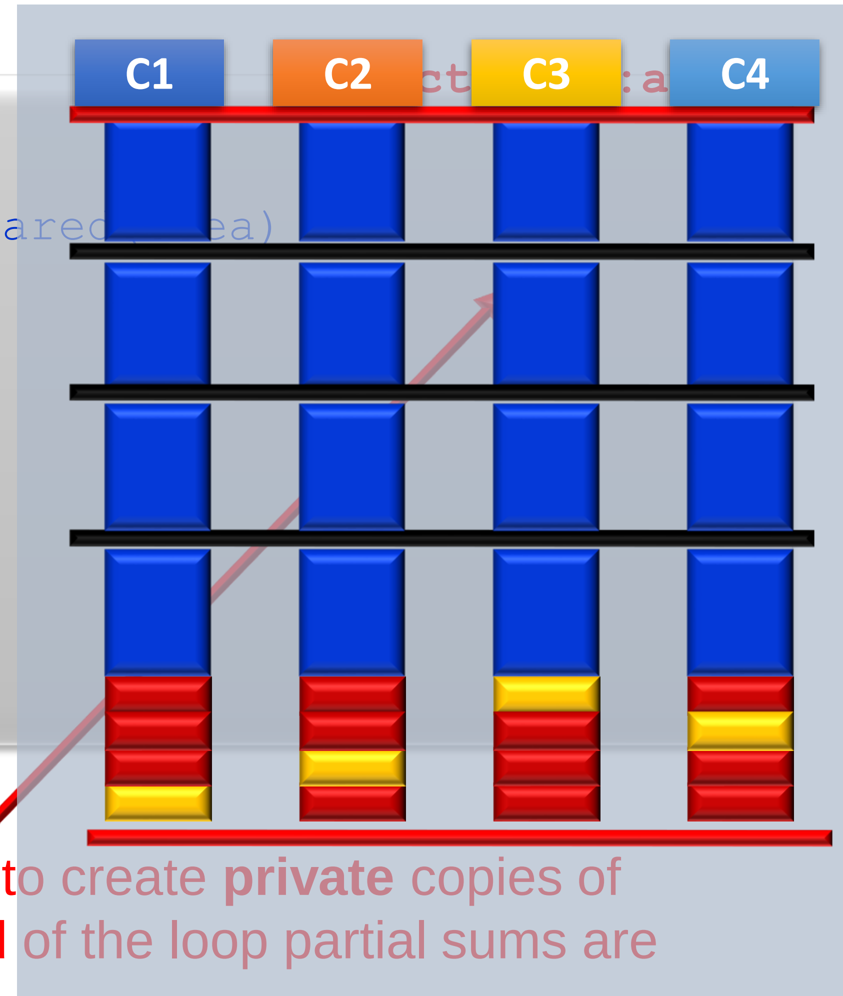
Each thread computes partial sums on private copies of the reduction variable

The r… er to create **private** copies of the **a**… nd of the loop partial sums are comb…

C ance!




```
doub
int
#pra                                   shared(   ea)
{
   fo
      
      a
   }
}
   pi
```



The r … er to create **private** copies of
the **ar** … nd of the loop partial sums are
comb…